

JAIN (DEEMED-TO-BE UNIVERSITY)

School of Computer Science and Engineering

Faculty of Engineering and Technology

ASSIGNMENT — UNIT III

Implementing a Continuous Integration (CI) Pipeline Using GitHub Actions

Course	24CSE538 — Cloud DevOps
Unit	Unit III — CI/CD and Automation Tools
Application	Student Result Management
Submission Mode	Individual
Technologies	Java, Maven, JUnit 5, Git, GitHub, GitHub Actions
Student Name	Bhupathiraju Adithya varma
USN	24BTRBT006
Branch	CSE-BLOCKCHAIN TECHNOLOGY

1. Objective

The objective of this practical is to implement a basic Continuous Integration (CI) pipeline for a Java application using Maven, JUnit, Git, GitHub and GitHub Actions. The pipeline is designed to automatically build and test the Student Result Management application whenever changes are pushed to the GitHub repository. The implementation demonstrates the complete CI flow: Code Change → Git Push → GitHub Repository → GitHub Actions → Ubuntu Runner → Maven Build → Automated Tests → CI Result.

2. Problem / Application Description

Student Result Management is a small Java application that represents the result-processing part of an academic system. A StudentResult object stores a student identifier, student name and subject-wise marks. The application provides five meaningful operations:

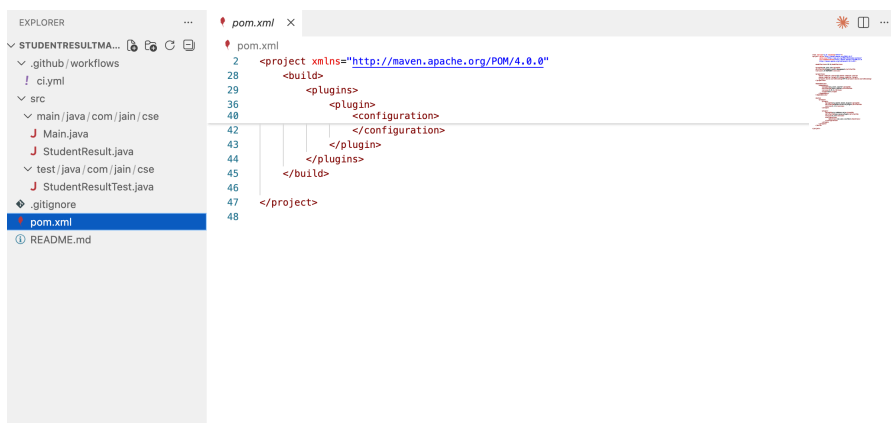
- Add marks for a subject.
- Calculate total marks.
- Calculate percentage.
- Calculate grade.
- Check whether the student has passed every registered subject.

The application also validates empty student information, empty subject names and marks outside the valid 0–100 range. Invalid input is reported using `IllegalArgumentException`.

3. Project Structure

```
student-result-management
├── .github
│   └── workflows
│       └── ci.yml
├── src
│   ├── main
│   │   ├── java
│   │   │   ├── com
│   │   │   │   ├── jain
│   │   │   │   │   ├── cse
│   │   │   │   │   └── StudentResult.java
│   │   └── test
│   │       ├── java
│   │       │   ├── com
│   │       │   │   ├── jain
│   │       │   │   │   ├── cse
│   │       │   │   │   └── StudentResultTest.java
│   ├── .gitignore
│   ├── pom.xml
│   └── README.md
```

The Java application is placed under `src/main/java`, JUnit tests under `src/test/java`, Maven configuration in `pom.xml`, and the GitHub Actions workflow in `.github/workflows/ci.yml`.



4. Java Source Code

The main class is StudentResult. The implementation provides five meaningful operations and validates invalid input before modifying the result.

```
package com.jain.cse;

import java.util.LinkedHashMap;
import java.util.Map;

public class StudentResult {    private
final String studentId;    private final
String studentName;    private final
Map<String, Double> marks;

    public StudentResult(String studentId, String studentName) {        if
(studentId == null || studentId.isBlank()) {            throw new
IllegalArgumentException("Student ID cannot be empty");        }        if
(studentName == null || studentName.isBlank()) {            throw new
IllegalArgumentException("Student name cannot be empty");        }

        this.studentId = studentId;
this.studentName = studentName;
this.marks = new LinkedHashMap<>();    }

    // Operation 1: Add marks for a subject.
    public void addMarks(String subject, double mark) {        if (subject == null
|| subject.isBlank()) {            throw new IllegalArgumentException("Subject
cannot be empty");        }        if (mark < 0 || mark > 100)
throw new IllegalArgumentException("Marks must be between 0 and
100");

        marks.put(subject, mark);
    }

    // Operation 2: Calculate total marks.
    public double calculateTotal()
    {
        return marks.values().stream()
.mapToDouble(Double::doubleValue)
.sum();
    }

    // Operation 3: Calculate percentage.
    public double calculatePercentage() {
    if (marks.isEmpty()) {        return
0.0;    }

    return calculateTotal() / marks.size();
    }

    // Operation 4: Calculate grade from the average percentage.
    public String calculateGrade() {
        double percentage = calculatePercentage();

        if (percentage >= 90) return "A+";
if (percentage >= 80) return "A";
if (percentage >= 70) return "B";
if (percentage >= 60) return "C";
if (percentage >= 50) return "D";
if (percentage >= 40) return "E";
return "F";    }

    // Operation 5: Check whether the student passed every registered subject.
    public boolean isPassed() {
        return !marks.isEmpty() && marks.values().stream().allMatch(mark -> mark >= 40);
    }

    public String getStudentId() {
    return studentId;    }

    public String getStudentName() {
    return studentName;    }
}
```

5 JUnit

JUnit 5 is used to verify the important application operations. Five test cases are included, which is above the assignment minimum of three. The tests cover total calculation, percentage calculation, grade calculation, pass/fail evaluation and invalid marks.

```
package com.jain.cse;

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;
```

```

class StudentResultTest {

    @Test
    void testCalculateTotal() {          StudentResult result = new
StudentResult("JAIN001", "Rahul");      result.addMarks("Java",
85);      result.addMarks("Cloud DevOps", 90);
result.addMarks("DBMS", 80);

        assertEquals(255.0, result.calculateTotal());
    }

    @Test
    void testCalculatePercentage() {          StudentResult result = new
StudentResult("JAIN002", "Ananya");      result.addMarks("Java",
80);      result.addMarks("Cloud DevOps", 90);
result.addMarks("DBMS", 70);

        assertEquals(80.0, result.calculatePercentage());
    }

    @Test
    void testCalculateGrade() {          StudentResult result = new
StudentResult("JAIN003", "Vikram");      result.addMarks("Java",
92);      result.addMarks("Cloud DevOps", 88);
result.addMarks("DBMS", 95);

        assertEquals("A+", result.calculateGrade());
    }

    @Test
    void testPassedResult() {          StudentResult result = new
StudentResult("JAIN004", "Priya");      result.addMarks("Java",
65);      result.addMarks("Cloud DevOps", 58);
result.addMarks("DBMS", 72);

        assertTrue(result.isPassed());
    }

    @Test
    void testInvalidMarks() {          StudentResult result = new
StudentResult("JAIN005", "Arun");

    assertEquals(
        () -> result.addMarks("Java", 105)
    );
}

```

6. Maven Configuration

The pom.xml file defines the Maven project coordinates, Java compiler level, JUnit 5 dependency and Maven Surefire plugin used to execute the tests.

```

<?xml version="1.0" encoding="UTF-8"?> <project xmlns="http://
maven.apache.org/POM/4.0.0"          xmlns:xsi="http://
www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
<modelVersion>4.0.0</modelVersion>

    <groupId>com.jain.cse</groupId>
    <artifactId>student-result-management</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>5.10.2</version>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-surefire-plugin</artifactId>
                <version>3.2.5</version>
            </plugin>
        </plugins>
    </build>
</project>

```

```

        </plugin>
    </plugins>
</build>
</project>

```

7. Local Maven Test Execution and Output

The required local verification should be performed from the project root using Maven. The primary command

is: `mvn clean test`

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
bhupathirajuadithyavarma@bhupathiraju-ka-MacBook-Air StudentResultManagement_GitHub_Project % mvn clean test
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/surefire-providers/3.2.5/surefire-providers-3.2.5.pom (2.6 kB at 58 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/common-java5/3.2.5/common-java5-3.2.5.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/junit/platform/junit-platform-launcher/1.9.3/junit-platform-launcher-1.9.3.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/common-java5/3.2.5/common-java5-3.2.5.pom (2.8 kB at 74 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/junit/platform/junit-platform-launcher/1.9.3/junit-platform-launcher-1.9.3.pom (3.0 kB at 80 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/junit/platform/junit-platform-engine/1.9.3/junit-platform-engine-1.9.3.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/junit/platform/junit-platform-engine/1.9.3/junit-platform-engine-1.9.3.pom (3.2 kB at 76 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/opentest4j/opentest4j/1.2.0/opentest4j-1.2.0.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/junit/platform/junit-platform-commons/1.9.3/junit-platform-commons-1.9.3.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/junit/platform/junit-platform-commons/1.9.3/junit-platform-commons-1.9.3.pom (2.8 kB at 81 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/opentest4j/opentest4j/1.2.0/opentest4j-1.2.0.pom (1.7 kB at 43 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/surefire-junit-platform/3.2.5/surefire-junit-platform-3.2.5.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/surefire-junit-platform/3.2.5/surefire-junit-platform-3.2.5.jar (27 kB at 646 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/common-java5/3.2.5/common-java5-3.2.5.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/junit/platform/junit-platform-launcher/1.9.3/junit-platform-launcher-1.9.3.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/junit/platform/junit-platform-engine/1.9.3/junit-platform-engine-1.9.3.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/opentest4j/opentest4j/1.2.0/opentest4j-1.2.0.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/junit/platform/junit-platform-commons/1.9.3/junit-platform-commons-1.9.3.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/common-java5/3.2.5/common-java5-3.2.5.jar (18 kB at 462 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/opentest4j/opentest4j/1.2.0/opentest4j-1.2.0.jar (7.7 kB at 159 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/junit/platform/junit-platform-commons/1.9.3/junit-platform-commons-1.9.3.jar (103 kB at 2.1 MB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/junit/platform/junit-platform-engine/1.9.3/junit-platform-engine-1.9.3.jar (189 kB at 3.9 MB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/junit/platform/junit-platform-launcher/1.9.3/junit-platform-launcher-1.9.3.jar (169 kB at 3.0 MB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/junit/platform/junit-platform-launcher/1.10.2/junit-platform-launcher-1.10.2.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/junit/platform/junit-platform-launcher/1.10.2/junit-platform-launcher-1.10.2.pom (3.0 kB at 80 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/junit/platform/junit-platform-launcher/1.10.2/junit-platform-launcher-1.10.2.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/junit/platform/junit-platform-launcher/1.10.2/junit-platform-launcher-1.10.2.jar (184 kB at 2.1 MB/s)
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.jain.cse.StudentResultTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.031 s -- in com.jain.cse.StudentResultTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO]
[INFO] -----
[INFO] Total time: 7.222 s
[INFO] Finished at: 2026-10-08T00:49:58+05:30
[INFO]

```

8. Git and GitHub Setup

Initialize Git in the project directory, create a GitHub repository, commit the complete project and push the main branch.

```
git init git
branch -M main
git add .
git commit -m "Initial Student Result Management CI project"
git remote add origin <YOUR_GITHUB_REPOSITORY_URL>
git push -u origin main
```

```
● bupathirajuadithyavarma@bupathiraju-ka-MacBook-Air StudentResultManagement_GitHub_Project % git init
Reinitialized existing Git repository in /Users/bupathirajuadithyavarma/Downloads/StudentResultManagement_GitHub_Project/.git/
● bupathirajuadithyavarma@bupathiraju-ka-MacBook-Air StudentResultManagement_GitHub_Project % git branch -M main
● bupathirajuadithyavarma@bupathiraju-ka-MacBook-Air StudentResultManagement_GitHub_Project % git add
Nothing specified, nothing added.
hint: Maybe you wanted to say 'git add .' ?
hint: Disable this message with "git config set advice.addEmptyPathsSpec false"
● bupathirajuadithyavarma@bupathiraju-ka-MacBook-Air StudentResultManagement_GitHub_Project % git add.
git: 'add.' is not a git command. See 'git --help'.

The most similar command is
  add
● bupathirajuadithyavarma@bupathiraju-ka-MacBook-Air StudentResultManagement_GitHub_Project % git add .
git commit -m "Initial Student Result Management CI project"
On branch main
nothing to commit, working tree clean
● bupathirajuadithyavarma@bupathiraju-ka-MacBook-Air StudentResultManagement_GitHub_Project % git remote -v
origin  https://github.com/bhadithya06-afk/StudentResultManagement_GitHub_Project.git (fetch)
origin  https://github.com/bhadithya06-afk/StudentResultManagement_GitHub_Project.git (push)
● bupathirajuadithyavarma@bupathiraju-ka-MacBook-Air StudentResultManagement_GitHub_Project % git push -u origin main
remote: Repository not found.
fatal: repository 'https://github.com/bhadithya06-afk/StudentResultManagement_GitHub_Project.git/' not found
● bupathirajuadithyavarma@bupathiraju-ka-MacBook-Air StudentResultManagement_GitHub_Project % git remote remove origin
● bupathirajuadithyavarma@bupathiraju-ka-MacBook-Air StudentResultManagement_GitHub_Project % git remote add origin https://github.com/bhadithya06-afk/StudentResultManagement_GitHub_Project.git
● bupathirajuadithyavarma@bupathiraju-ka-MacBook-Air StudentResultManagement_GitHub_Project % git remote -v
origin  https://github.com/bhadithya06-afk/StudentResultManagement_GitHub_Project.git (fetch)
origin  https://github.com/bhadithya06-afk/StudentResultManagement_GitHub_Project.git (push)
● bupathirajuadithyavarma@bupathiraju-ka-MacBook-Air StudentResultManagement_GitHub_Project % git push -u origin main
remote: Repository not found.
fatal: repository 'https://github.com/bhadithya06-afk/StudentResultManagement_GitHub_Project.git/' not found
● bupathirajuadithyavarma@bupathiraju-ka-MacBook-Air StudentResultManagement_GitHub_Project % █
```

9. GitHub Repository

The repository should contain the Java source code, JUnit test code, pom.xml, README.md and .github/workflows/ci.yml. Host it in the following public GitHub repository:

https://github.com/bhadithya06-afk/StudentResultManagement_GitHub_Project

The screenshot shows a GitHub repository page. At the top, the repository name is 'bhadithya06-afk / StudentResultManagement_GitHub_Project'. Below this, there's a table of files and folders. The files include '.github/workflows', 'src', '.gitignore', 'README.md', and 'pom.xml', all committed 10 minutes ago. Below the file list, the 'README' tab is selected, showing the title 'Student Result Management' and a description: 'A small Java and Maven application created for the Unit III Cloud DevOps assignment.' The 'Features' section lists: 'Add marks for different subjects', 'Calculate total marks', 'Calculate percentage', 'Calculate grade', 'Check pass/fail status', and 'Validate marks between 0 and 100'. The 'Technologies' section is also visible. On the right side, there's a sidebar with 'About' (No description, website, or topics provided), 'Releases' (No releases published), 'Packages' (No packages published), 'Contributors' (1 contributor: bhadithya06-afk), and 'Languages' (Java 100%).

File/Folder	Commit Message	Time
.github/workflows	Initial Student Result Management CI project	10 minutes ago
src	Initial Student Result Management CI project	10 minutes ago
.gitignore	Initial Student Result Management CI project	10 minutes ago
README.md	Initial Student Result Management CI project	10 minutes ago
pom.xml	Initial Student Result Management CI project	10 minutes ago

Student Result Management

A small Java and Maven application created for the Unit III Cloud DevOps assignment.

Features

- Add marks for different subjects
- Calculate total marks
- Calculate percentage
- Calculate grade
- Check pass/fail status
- Validate marks between 0 and 100

Technologies

10. GitHub Actions Workflow

The workflow is stored at `.github/workflows/ci.yml`. It is triggered on every push, uses an Ubuntu-based runner, checks out the repository, configures Java 17 using Temurin and executes `mvn clean test`.

```
name: Student Result Management CI

on:
  push:

jobs:
  build-and-test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Set up Java
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'
          cache: maven

      - name: Build and Test
        run: mvn clean test
```

The screenshot displays the GitHub Actions interface for the `StudentResultManagement_GitHub_Project`. The workflow `Student Result Management CI` is shown, with the `build-and-test` job highlighted. The job status is **Completed**, and the workflow is **Successful**. The job steps are listed with their durations:

- Set up job: 2s
- Checkout Repository: 1s
- Set up Java: 2s
- Build and Test: 5s**
- Post Set up Java: 0s
- Post Checkout Repository: 0s
- Complete job: 0s

The `Build and Test` step is expanded, showing a search bar for logs and a list of steps with their durations.

Summary

All jobs

build-and-test

Run details

Usage

Workflow file

Triggered via push 6 minutes ago

Status

Total duration

👤 bhadithya06-afk pushed

↻ a427e8a

main

Success

16s

Artifacts

—

ci.yml

on: push

✓ build-and-test

13s

🔍

—

+

Annotations

2 warnings and 1 notice

⚠️ build-and-test

Node.js 20 is deprecated. The following actions target Node.js 20 but are being force...
[Show more](#)

⚠️ build-and-test

setup-java v4 is deprecated and will no longer receive updates. Please migrate to act...
[Show more](#)

ℹ️ build-and-test

"The ubuntu-latest label will migrate to Ubuntu 26 beginning October 19, 2026. For mo...
[Show more](#)

11. CI Pipeline Execution

After the workflow file is committed and pushed, open the Actions section of the GitHub repository. The Student Result Management CI workflow should start automatically because the workflow is configured with a push trigger. The expected sequence is: checkout repository and set up Java and run Maven clean test and report success or failure.

```

1 ▶ Run mvn clean test
2 [INFO] Scanning for projects...
3 [INFO]
4 [INFO] -----< com.jain.cse:student-result-management >-----
5 [INFO] Building student-result-management 1.0-SNAPSHOT
6 [INFO] from pom.xml
7 [INFO] -----[ jar ]-----
8 [INFO] Loaded 24024 auto-discovered prefixes for remote repository central (prefixes-central.txt)
9 [INFO] Loaded 74 auto-discovered prefixes for remote repository apache.snapshots (prefixes-apache.snapshots.txt)
10 [INFO]
11 [INFO] --- clean:3.5.0:clean (default-clean) @ student-result-management ---
12 [INFO]
13 [INFO] --- resources:3.5.0:resources (default-resources) @ student-result-management ---
14 [INFO] skip non existing resourceDirectory
15 /home/runner/work/StudentResultManagement_GitHub_Project/StudentResultManagement_GitHub_Project/src/main/resources
16 [INFO]
17 [INFO] --- compiler:3.16.0:compile (default-compile) @ student-result-management ---
18 [INFO] Recompiling the module because of changed source code.
19 [INFO] Compiling 2 source files with javac [debug target 17] to target/classes
20 [INFO]
21 [INFO] --- resources:3.5.0:testResources (default-testResources) @ student-result-management ---
22 [INFO] skip non existing resourceDirectory
23 /home/runner/work/StudentResultManagement_GitHub_Project/StudentResultManagement_GitHub_Project/src/test/resources
24 [INFO]
25 [INFO] --- compiler:3.16.0:testCompile (default-testCompile) @ student-result-management ---
26 [INFO] Recompiling the module because of changed dependency.
27 [INFO] Compiling 1 source file with javac [debug target 17] to target/test-classes
28 [INFO]
29 [INFO] --- surefire:3.2.5:test (default-test) @ student-result-management ---
30 [INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnit4PlatformProvider
31 [INFO]
32 [INFO] -----
33 [INFO] T E S T S
34 [INFO] -----
35 [INFO] Running com.jain.cse.StudentResultTest

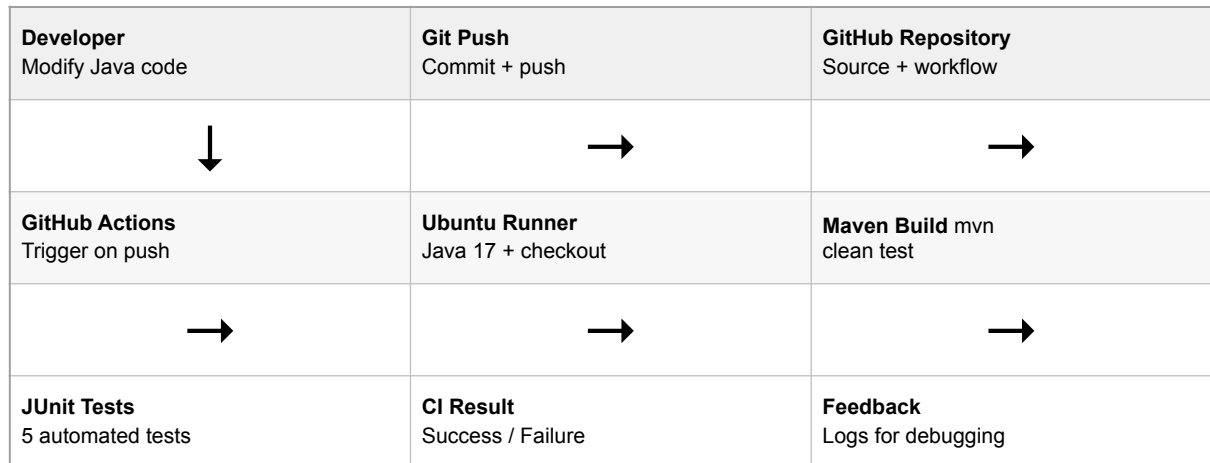
```

```

39 [INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.036 s -- in com.jain.cse.StudentResultTest
40 [INFO]
41 [INFO] Results:
42 [INFO]
43 [INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
44 [INFO]
45 [INFO] -----
46 [INFO] BUILD SUCCESS
47 [INFO] -----
48 [INFO] Total time: 3.403 s

```

12. CI Pipeline Diagram



The implemented CI pipeline follows the required flow from developer code change through Git push, GitHub, GitHub Actions and an Ubuntu runner, followed by Maven build/test execution and a final success or failure result.

13. Short Answer Questions

1. What is Continuous Integration?

Continuous Integration is a development practice in which developers frequently integrate code changes into a shared repository and an automated process builds and tests those changes. In this project, GitHub Actions performs the automated Maven build and JUnit test execution after a push.

2. What is the purpose of pom.xml in a Maven project?

pom.xml is Maven's project configuration file. It defines the project information, Java compiler settings, dependencies such as JUnit, and build plugins required for the project.

3. Why is a YAML workflow file required for GitHub Actions?

The YAML workflow file describes the CI automation. It specifies when the workflow should run, which runner should execute it, which actions should be used, how Java should be configured and which Maven commands should be executed.

4. What is the purpose of a GitHub Actions runner?

A runner is the machine environment that executes the workflow steps. In this project, the Ubuntu-based runner checks out the repository, prepares Java 17 and runs the Maven build and tests.

5. What happens when an automated test fails inside the CI pipeline?

Maven returns a failure status, the GitHub Actions job is marked as failed and the workflow logs show the failed test information. This provides immediate feedback so the developer can correct the code before considering the change successfully integrated.

6. Differentiate between pom.xml and the GitHub Actions YAML workflow.

pom.xml controls the Maven project, including dependencies, compiler configuration and build plugins. The GitHub Actions YAML controls CI automation, including the trigger, runner, setup steps and commands executed by the runner.

7. Explain the complete flow of your CI pipeline from a code change to the final CI result.

A developer changes the Java application, commits the change and pushes it to GitHub. The push triggers the CI workflow. GitHub Actions creates an Ubuntu runner, checks out the repository, sets up Java 17 and executes

mvn clean test. Maven builds the project and JUnit runs the automated tests. If the tests pass, the workflow finishes successfully; if a test fails, the workflow reports failure and provides logs for diagnosis.

8. Why is automated testing important in a CI pipeline?

Automated testing provides a repeatable verification step whenever code changes are integrated. It helps detect regressions early, reduces manual testing effort and provides quick feedback about whether the application still builds and its important operations behave as expected.

14. Conclusion

The Student Result Management application demonstrates a Continuous Integration pipeline using Java, Maven, JUnit 5, Git, GitHub and GitHub Actions. The application provides multiple meaningful result-processing operations and five automated JUnit test cases. Maven manages the build and test process, while GitHub Actions automatically repeats that process after a Git push. The workflow uses an Ubuntu-based runner and Java 17, and the CI result is determined by the Maven test outcome. This project demonstrates CI only and does not include deployment or Continuous Delivery/Continuous Deployment.

Github repository:

https://github.com/bhadithya06-afk/StudentResultManagement_GitHub_Project